SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	J Pranitha	**Reg. No.:**	192472073
Section / Batch:	B. Tech (AI & DS)	**Date:**	14.07.2026

Code of Conduct

I _J Pranitha 192472073__ (Name / Reg No) certify that this submission is my original work and that I have adhered tothe guidelines specified for this assessment. I understand that any violation of academic integrity rules will result indisciplinary action.

Signature of the Student: ___Pranitha __

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in thecoding challenge using appropriate algorithms, data structures, and programming practices. • Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and outputto verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exceptionhandling (where applicable), and upload the source code, sample outputs, and README file to the designatedGitHub repository.
- Duration: 40 minutes.

Regular Expression Pattern Matching	Regular Expressions, Basic Regular ExpressionPatterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular ExpressionPatterns	3

Annexure A – Coding Challenge

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system mustvalidate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules

Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)

Mobile Number

Must contain exactly 10 digits.
First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

ANSWER:-

```
import re email_pattern =
r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'
mobile_pattern = r'^[6-9]\d{9}$' print("VALID INPUT") email =
"abc_123@gmail.com" password = "Abc@1234" mobile = "9876543210"
print("Email:", email) if re.fullmatch(email_pattern, email): print("Valid
Email") else: print("Invalid Email") print("Password:", password) if
re.fullmatch(password_pattern, password): print("Strong Password")
else:
    print("Weak Password")
print("Mobile Number:", mobile) if
re.fullmatch(mobile_pattern, mobile):
```

```
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
print("\nINVALID INPUT")
email = "labc@gmail"
password = "abc123"
mobile = "5123456789"
print("Email:", email)
if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")
print("Password:", password)
if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
print("Mobile Number:", mobile)
if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

```
==== RESTART: C:/Users/bvbvi/AppData/Local/Programs/Python/P
ython313/zvsv.py ===
VALID INPUT
Email: abc_123@gmail.com
Valid Email
Password: Abc@1234
Strong Password
Mobile Number: 9876543210
Valid Mobile Number

INVALID INPUT
Email: labc@gmail
Invalid Email
Password: abc123
Weak Password
Mobile Number: 5123456789
Invalid Mobile Number
```

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description

▪ States

▪ Input alphabet

▪ Transition table

▪ Initial state

▪ Final state(s)

- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

ANSWER:-

```
states = ['q0', 'q1', 'q2']

alphabet = ['a', 'b']

transition = {
    ('q0', 'a'): 'q1',
    ('q0', 'b'): 'q0',
    ('q1', 'a'): 'q1',
    ('q1', 'b'): 'q2',
    ('q2', 'a'): 'q1',
    ('q2', 'b'): 'q0'
}


initial_state = 'q0'

final_states = ['q2']


print("VALID INPUT")
string = "abaab"

current_state = initial_state
path = [current_state]

for symbol in string: current_state =
transition[(current_state, symbol)]
path.append(current_state)

print("Input String:", string)
print("Transition Path:", " → ".join(path))

if current_state in
final_states:
print("Accepted") else:
print("Rejected")

print("\nINVALID INPUT")
string = "ababa"

current_state = initial_state
path = [current_state]
```

```
for symbol in string:
    current_state = transition[(current_state, symbol)]
    path.append(current_state)

print("Input String:", string)
print("Transition Path:", " → ".join(path))

if current_state in final_states:
    print("Accepted")
else:
    print("Rejected")
```

```
= RESTART: C:/Users/bvbvi/AppData/Local/Programs/
on313/zvsv.py
VALID INPUT
Input String: abaab
Transition Path: q0 → q1 → q2 → q1 → q1 → q2
Accepted

INVALID INPUT
Input String: ababa
Transition Path: q0 → q1 → q2 → q1 → q2 → q1
Rejected
```

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix

6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

ANSWER:-

```
import re

text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
machine learning
"""

print("Sample Text:")
print(text)

print("Menu") print("1. Search
Date") print("2. Search Phone
Number") print("3. Search
Hashtag") print("4. Search
Mention") print("5. Search
Prefix") print("6. Search Suffix")
print("7. Search Word")

choice = int(input("Enter your choice: "))

if choice == 1:
    result = re.findall(r'\b\d{2}/\d{2}/\d{4}\b', text)
print("Date Found:", result)

elif choice == 2:
    result = re.findall(r'\b[6-9]\d{9}\b', text)
print("Phone Number Found:", result)

elif choice == 3:
    result = re.findall(r'#\w+', text)
    print("Hashtag Found:", result)

elif choice == 4: result =
re.findall(r'@\w+', text)
    print("Mention Found:", result)

elif choice == 5: prefix =
input("Enter Prefix: ")
    result = re.findall(r'\b' + prefix + r'\w*', text)
print("Matching Words:", result)

elif choice == 6: suffix =
input("Enter Suffix: ")
    result = re.findall(r'\b\w*' + suffix + r'\b', text)
print("Matching Words:", result)

elif choice == 7: word =
input("Enter Word: ")
```

```
xt)

:/bvbvi/AppData/Local/Programs

)26

:ocessing

ıber

2
  ['9876543210']
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary
Problem Understanding	20%	Clearly understands problem constraints, edgecases, and competitive requirements
Algorithm	25%	Selects optimal algorithm with besttime and space complexity for competitive constraints
Code Implementation	20%	Writes correct, efficient, and cleancode that passesall constraints

Competitive Performance	20%	Solves problem within time limits and handles largeinputs effectively	Solves within acceptable time with minor delays	Struggles with timelimits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hiddencases	Handles most test cases	Misses importantedge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementati
1			
2			
3			

Faculty In-charge	Course Coord
Signature: _____	Signature: __
Date: _____	Date: _____